Geometry for Regionals

Exact when you can, careful when you can't

Learning goals

- Build a small, tested geometry core around the **cross product**
- Decide segment intersection and point-in-polygon **exactly** with integers
- Compute convex hulls, areas, lattice-point counts
- Intersect lines in floating point without precision disasters
- Know the standard precision traps and their fixes

Why teams skip geometry (and shouldn't)

- Regionals usually have 1–2 geometry problems
- Many are *medium*: hull, polygon area, segment tests, a sweep
- They are skipped because of fear of special cases and precision
- A tested notebook core + integer arithmetic removes most of the fear

The core: one struct, one product

```
struct P {
    ll x, y;
    P operator-(const P& o) const { return {x - o.x, y - o.y}; }
    bool operator<(const P& o) const { return x != o.x ? x < o.x : y < o.y; }
};
ll cross(P a, P b) { return a.x * b.y - a.y * b.x; }
ll cross(P o, P a, P b) { return cross(a - o, b - o); }
ll dot(P a, P b) { return a.x * b.x + a.y * b.y; }
```

- Integer input ⇒ keep **everything** integer as long as possible
- $|\text{coords}| \le 10^9$ ⇒ cross of differences up to $8 \cdot 10^{18}$: fits `long long` (barely)

Orientation

```
cross(o, a, b) > 0   b is LEFT of o→a   (counter-clockwise turn)
cross(o, a, b) < 0   b is RIGHT         (clockwise)
cross(o, a, b) = 0   collinear
```

- Area of triangle = |cross| / 2
- Almost every predicate below reduces to signs of cross products
- *CSES 2189 Point Location Test*

Segment intersection (exact)

```
bool segmentsIntersect(P a, P b, P c, P d) {
    int o1 = orient(a, b, c), o2 = orient(a, b, d);
    int o3 = orient(c, d, a), o4 = orient(c, d, b);
    if (o1 * o2 < 0 && o3 * o4 < 0) return true;      // proper crossing
    if (o1 == 0 && onSegment(a, b, c)) return true;   // touching or
    if (o2 == 0 && onSegment(a, b, d)) return true;   // collinear
    if (o3 == 0 && onSegment(c, d, a)) return true;   // overlap
    if (o4 == 0 && onSegment(c, d, b)) return true;
    return false;
}
```

- `onSegment` : collinear point inside the bounding box
- *CSES 2190*, Kattis `segmentintersection` (needs the point too)

Line intersection point

- Lines `A + t·r` and `C + u·s`
- `t = cross(C − A, s) / cross(r, s)`
- `cross(r, s) = 0`: parallel; also `cross(C − A, r) = 0`: same line

```
ll den = cross(r, s);
if (den == 0) return /* parallel or same */;
long double t = (long double)cross(C - A, s) / den;
return {A.x + t * r.x, A.y + t * r.y};
```

- Decide degeneracies with **integers**; only the final point is floating

Polygon area (shoelace)

```
2A = Σ cross(p_i, p_{i+1})      (indices mod n)
```

- Sign: positive ⟺ counter-clockwise order
- Integer coordinates ⇒ 2A is an integer → print `A` exactly as `x` or `x.5`
- *CSES 2191 Polygon Area* prints 2A directly

Lattice points: Pick's theorem

- Simple polygon with integer vertices:
  $2A = 2I + B - 2$
- `B` = Σ over edges of gcd(|dx|, |dy|)
- So `I = (2A − B + 2) / 2`

```
twiceArea += x[i] * y[j] - x[j] * y[i];
boundary  += gcd(llabs(x[j] - x[i]), llabs(y[j] - y[i]));
```

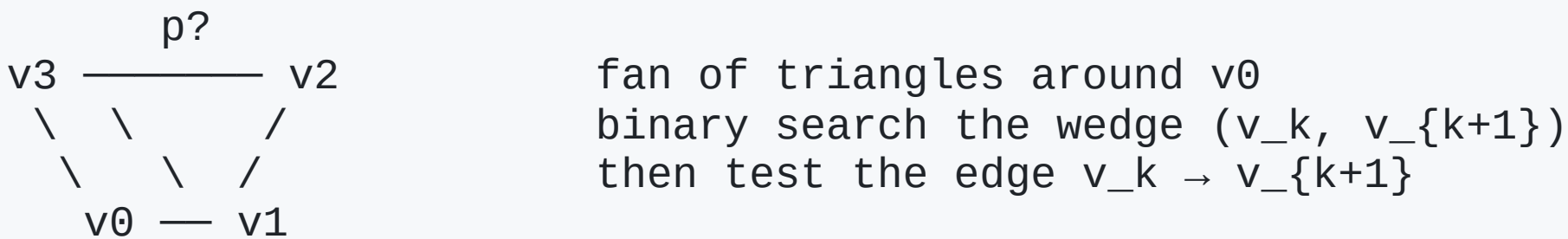
- *CSES 2193 Polygon Lattice Points*

Point in polygon (any simple polygon)

```
int pointInPolygon(const vector<P>& poly, P p) {   // 0 out, 1 in, 2 boundary
    bool inside = false;
    for (int i = 0, n = poly.size(); i < n; i++) {
        P a = poly[i], b = poly[(i + 1) % n];
        if (onSegment(a, b, p)) return 2;
        if (a.y > b.y) swap(a, b);
        if (a.y <= p.y && p.y < b.y && cross(b - a, p - a) > 0)
            inside = !inside;
    }
    return inside ? 1 : 0;
}
```

- Half-open rule `a.y ≤ p.y < b.y` handles rays through vertices
- O(n) per query, exact

Point in convex polygon: O(log n)



- Reject if p is right of `v0→v1` or left of `v0→v_{n−1}`
- Binary search the largest k with `cross(v0, v_k, p) ≥ 0`
- Boundary on the fan's first/last edge is a special case
- *CF 166B Polygons*: is polygon B strictly inside convex A?

Convex hull: monotone chain

```
sort(pts.begin(), pts.end()); /* + unique */
for (int i = 0; i < n; i++) {                          // lower
    while (k >= 2 && cross(h[k-2], h[k-1], pts[i]) <= 0) k--;
    h[k++] = pts[i];
}
for (int i = n - 2, lo = k + 1; i >= 0; i--) {         // upper
    while (k >= lo && cross(h[k-2], h[k-1], pts[i]) <= 0) k--;
    h[k++] = pts[i];
}
h.resize(k - 1);                                        // CCW, no collinear
```

- `<= 0` drops collinear points; keeping them (CSES 2195) needs `< 0` **and** care when all points are collinear

Hull checklist

- Duplicates removed? (`unique` after sorting)
- $n \le 2$ or all collinear: what must be printed?
- Orientation and starting vertex required by the output?
- Collinear boundary points: kept or dropped?
- Kattis `convexhull` vs CSES 2195 differ on exactly these points

Common hull applications

- Farthest pair (diameter): rotating calipers on the hull (National track)
- Minimum perimeter fence around points
- "Which points can be seen from infinity?"
- Upper hull of lines/points ⇒ convex hull trick (National track)
- Smallest enclosing rectangle / width

Precision: the rules

1. Integer input → integer predicates (cross, dot, squared distances)
2. Compare **squared** distances; avoid `sqrt` until printing
3. Need floats? Use `long double` and an epsilon **relative to the scale**
4. Never test `x == y` on doubles; use `fabsl(x - y) < EPS`
5. Avoid angles; if needed, sort by **cross product**, not `atan2`

Angle sorting without atan2

```
int half(P p) { return p.y < 0 || (p.y == 0 && p.x < 0); }  // 0: [0, π), 1: [π, 2π)
bool angleLess(P a, P b) {
    if (half(a) != half(b)) return half(a) < half(b);
    return cross(a, b) > 0;
}
```

- Exact for integer vectors (no zero vector)
- *CF 598C Nearest vectors* is the famous trap: `atan2` in `double` fails; `long double` or exact comparison passes

Circles (the usual suspects)

- Point in circle: `dist² ≤ r²` (integers!)
- Line–circle intersection: project center, Pythagoras
- Circle through 3 points (circumcenter): solve 2×2 linear system
- *CF 1C Ancient Berland Circus*: circumcircle + find the smallest regular polygon (floating gcd of angles) — a classic precision exercise

Complexity

Task	Time
Orientation, segment test	$O(1)$
Polygon area, Pick	$O(n)$
Point in polygon	$O(n)$ per query
Point in convex polygon	$O(\log n)$ per query
Convex hull	$O(n \log n)$

Common pitfalls

- `int` cross products → overflow
- Ray casting counting a vertex twice (no half-open rule)
- Hull with duplicates or all-collinear input
- `atan2` / `acos` near ±1 → NaN or wrong order
- Printing `-0.000`
- EPS too large for small coordinates, too small for large

Practice problems

- CSES 2189 Point Location Test · 2190 Line Segment Intersection
- CSES 2191 Polygon Area · 2192 Point in Polygon · 2193 Polygon Lattice Points
- CSES 2195 Convex Hull (collinear points kept)
- Kattis `convexhull`, `pointinpolygon`, `polygonarea`, `segmentintersection`
- Codeforces 166B — Polygons
- Codeforces 598C — Nearest vectors
- Codeforces 1C — Ancient Berland Circus

Summary

- Cross product = orientation = area; build everything on it
- Integer inputs: exact predicates, floating point only for outputs
- Ray casting with the half-open rule; O(log n) for convex polygons
- Monotone chain hull; decide collinear/duplicate policy explicitly
- Next: strings for regionals